IP Model Automation

Project Report — Pipeline, Modeled IPs, and Integration Workloads

2026-07-06 | 11 modeled artifacts (9 IPs + 2 subsystems) | 58 tests | all gates green

1. Introduction and Goals

This project turns hardware IP design-level documents (DLDs) — which vary in format and often omit details — into validated, executable SimPy transaction-level performance models with unit tests. The central idea is a normalized YAML template that sits between design intent and generated code: the DLD is the only source for authoring the template, and the reviewed (promoted) template is the only source of truth for model generation. Anything the DLD does not state is surfaced in a gaps report with a labeled conservative default — never silently invented. Every stage of the flow is protected by an automated, deterministic gate, and a single command (tools/validate_dld_flow.py) proves the entire repository is consistent.

The system was designed so that either a human engineer or an LLM agent can do the judgment work (reviewing templates, implementing models) inside a strict machine-checked contract, while deterministic tools handle everything that can be automated.

2. The Complete System — Components and How They Compose

The repository is organized as a closed loop: human design intent enters as a DLD, deterministic tools extract and check a template, an agent (human or LLM) fills the judgment gaps under a contract, and gates verify each artifact before the next stage may consume it.

Component	Location	Role in the system
DLDs	docs/*_dld.md	Human design intent. Authoring source only — 12 sections per document (Purpose, Scope, Interfaces, FSMs, timing table, Open Items) following conventions the extractor parses.
Schema	schemas/ip_model_template.schema.json	Machine-checkable JSON-Schema contract for templates: required sections, FSM/transition shapes, timing and scenario structures.
Templates	templates/*.template.yaml	The golden references — one reviewed template per artifact, the single source of truth for generation. templates/reference_template.yaml is an annotated, lint-clean authoring reference (excluded from discovery).
Tools	tools/ (11 scripts)	Deterministic extraction, linting, coverage, wiring, scaffolding, prompt packs, and validation. Dependency-light; several are text-based parsers so they run anywhere.
Skill	skills/ip-model-generation/	Instructions for the human/LLM doing the judgment work: SKILL.md defines the Stage 0 and Stage 1 workflows; references/ hold DLD extraction rules and model generation rules.
Harness	harness/ip_generation_loop.yaml	Declarative orchestration loop: 9 named stages from parse_dld to full_validation, a loop policy (max 3 iterations, stop on first pass, 8 fail conditions), and explicit pass criteria.
Agent contract	agents/ip_model_generation_agent.md	Binding contract for any agent implementing a model: inputs, Stage 0/1 responsibilities, forbidden actions (no invented behavior, no per-IP folders, no SystemC), and the completion report format.
Prompt packs	prompt_packs/ (generated)	Self-contained LLM generation requests produced from a reviewed template — the template content plus generation rules, so an external model needs nothing else.

Component	Location	Role in the system
Models	src/ip_model_automation/	Flat, one-file-per-artifact SimPy delay models; one class <CamelName>Model, one SimPy process per template FSM, IP-tagged logging, metrics dictionaries.
Tests	tests/ (58 tests)	Per-artifact unit tests derived from template test_scenarios, plus workflow tests that pin the registry, file layout, counts, tool behavior, and the reference template.

2.1 How the pieces work together

- The skill tells the agent WHAT to do (workflows, ordering, rules); the agent contract tells it what it MUST and MUST NOT do; the harness encodes WHEN each step runs and what counts as done; the schema and linter verify the artifact independently of who produced it.
- The harness's stages alternate deterministic commands (extraction, lint, coverage, scaffold, prompt pack, validation) with two agent stages (review_template, agent_implementation) — the only places judgment enters, both bounded by gates on either side.
- The loop policy makes generation self-correcting: up to 3 iterations, stopping on first pass, with explicit fail conditions (unresolved TODO_REVIEW, uncovered DLD FSM, scaffold shape error, test failure...). tools/run_loop_validation.py runs these gates deterministically and tools/inspect_harness.py reports the harness state.
- Everything converges in tools/validate_dld_flow.py: for all 11 DLDs it checks template existence, lint, strict DLD coverage, and subsystem wiring, then delegates to tools/validate_ip_flow.py for scaffold smoke tests, FSM scenario coverage, prompt packs, and the unit test suite.

3. The Generation Pipeline

```
DLD markdown (docs/<name>_dld.md)
  -> Stage 0a  dld_to_template.py      draft template + gaps report
  -> Stage 0b  review TODO_REVIEW      agent fills DLD-stated behavior only
  -> Stage 0c  lint + coverage --strict promote to templates/<name>.template.yaml
  -> Stage 1a  generate_model_scaffold  SimPy skeleton with FSM process stubs
  -> Stage 1b  agent implementation     model + tests from template only
  -> Stage 1c  validate_dld_flow.py     full front-end + model/test gate
```

Stage 0 (DLD to reviewed template): the extractor parses the DLD's conventions — FSM headings ('### N.M <Name> FSM') snake-case to exact template FSM names, the 'Total FSM/processes: N' line becomes fsm_count, and the timing table rows become per-FSM operations. Fields it cannot derive are emitted as TODO_REVIEW markers plus a gaps report. The reviewer resolves every marker using only DLD-stated behavior; the strict coverage gate then proves the promoted template matches the DLD's FSMs exactly and contains no leftover markers.

Stage 1 (template to model + tests): a scaffold generator emits the model skeleton; the agent fills behavior using only template fields (timing from timing_model, structure from fsm_processes/queues/resources, assertions from test_scenarios), then the full validation flow must pass.

3.1 Validation gates

Gate	Tool	What it proves
Template lint	template_lint.py	Full generation contract present: FSMs with states/transitions/queues/resources, per-FSM timing, scenario coverage of every FSM, per-IP contracts (e.g. arbitration bursts).

Gate	Tool	What it proves
DLD coverage (strict)	check_template_coverage.py	Template FSM names and fsm_count match the DLD exactly; no TODO_REVIEW remains. Interfaces are report-only (templates may consolidate).
Subsystem wiring	check_subsystem_wiring.py	Subsystem members exist (template + model + class), the subsystem model instantiates them, every connection endpoint is a real member API or glue FSM.
Scaffold smoke	validate_ip_flow.py	Every template generates a scaffold with the expected class and FSM process methods.
FSM test coverage	report_model_coverage.py	Every FSM appears in at least one template test scenario.
Unit tests	unittest discover (58 tests)	Functional and timing behavior of all 11 models, plus workflow/layout/registry invariants.
Top gate	validate_dld_flow.py	All of the above for all 11 DLDs in one command.

4. Modeled IPs (9)

Each IP is a flat SimPy model with one process per template FSM, template-driven delays, IP-tagged logging (log_level/log_file constructor args), and a metrics dictionary the unit tests assert on.

4.1 arbitration_ip — hierarchical command arbitration (3 FSMs)

- FSMs: arbiter_main (port/tenant/SQ weighted-RR scan over pending bitmaps), issue_pipeline (pending-count read, burst read, min-burst issue calculation, burst debit), policy_update (RR pointer and age updates).
- Models dual/single port topologies, per-level WRR weights, device/tenant/SQ burst credit limits, and downstream backpressure (set_issue_ready).
- Key metrics: grants, issued_commands, downstream_requests, burst_stalls, output_backpressure_cycles.

4.2 completion_ip — QoS-token completion scheduling (3 FSMs)

- FSMs: accept (tenant-checked intake), completion_scheduler (WRR tenant select, token check, debit, emit), refill (windowed token restore).
- Models per-tenant read/write/bandwidth token buckets, WWV write accounting (writes also cost read tokens at a configurable ratio), SOFT (burst) vs HARD limits, and output backpressure.
- Key metrics: completed_commands, token_stalls, output_stalls, burst_debits, refill_windows.

4.3 gdma_ip — descriptor-driven DMA engine (9 FSMs)

- FSMs: channel_control, descriptor_fetch, channel_scheduler, read_issue, read_response, write_issue, write_response, completion_update, interrupt_coalescing.
- Models descriptor validation, bounded descriptor/data buffers, source/destination/completion memory readiness stalls, and coalesced completion interrupts.
- Key metrics: descriptors_fetched, read/write_requests, buffer_occupancy, read/write/completion stalls, irq_count, last_latency.

4.4 timer_ip — system timer block (8 FSMs)

- FSMs: register_access, configuration_synchronizer, prescaler, counter, compare_event, interrupt_aggregation, watchdog, debug_freeze.

- Models shadowed configuration, prescaled counting, compare/overflow events, watchdog timeout (CRITICAL log), and debug-freeze gating.

4.5 axi_interconnect_ip — AXI fabric router (9 FSMs)

- FSMs: write_address_ingress, write_data_ingress, write_join_ordering, slave_write_arbitration, write_response_routing, read_address_ingress, slave_read_arbitration, read_data_routing, decode_error_response.
- Models address decode against configured routes, AW/W join, outstanding-transaction credit (stalls at the limit), per-slave readiness backpressure, and DECERR responses.
- Key metrics: write_joins, read/write grants, outstanding_stalls, slave stalls, decode_errors, read/write latency.

4.6 interrupt_controller_ip — interrupt delivery fabric (9 FSMs)

- FSMs: source_sampling, pending_update, mask_enable_filter, priority_resolver, cpu_delivery, acknowledge, end_of_interrupt, register_access, software_interrupt.
- Models edge and level triggers, enable/mask filtering, priority resolution, CPU delivery/ACK/EOI with level reassert at EOI, and software-injected interrupts.
- Key metrics: source_events, pending_updates, masked_irqs, delivered_irqs, ack_count, eoi_count, reasserted_level_irqs.

4.7 mailbox_ip — inter-processor messaging (5 FSMs)

- FSMs: register_access, message_push, message_pop, doorbell, interrupt_notify.
- Models per-channel bounded FIFOs with drop-on-full, doorbell events per enqueue, per-channel interrupt masking, and write-one-to-clear doorbell status.
- Key metrics: enqueued/delivered messages, dropped_on_full, doorbells, interrupt_count, masked_doorbells.

4.8 spi_master_ip — SPI serial controller (5 FSMs)

- FSMs: register_access, transmit_engine, shift_engine, receive_engine, interrupt_control.
- Models TX/RX byte FIFOs, per-bit shift timing with chip-select framing, RX overflow counting, and masked transfer-done interrupts.

4.9 i3c_ip — MIPI I3C master with in-band interrupts (5 FSMs)

- FSMs: register_access, command_engine, bit_transfer_engine, ibi_engine, interrupt_control.
- Models a queued private read/write command engine with open-drain bus arbitration, bit-level SDR shifting with ACK/NACK sampling, and I3C's signature feature: in-band interrupt (IBI) detection during bus-idle windows, arbitration, and payload capture. The command and IBI engines share the bus via a busy flag.
- Key metrics: commands_issued, transmitted/received bytes, ibi_requests/accepted/bytes_captured, interrupt_count, masked_interrupts.

5. Integration Workloads — Connected Subsystems (2)

Beyond isolated IP models, the project models combinations of IPs as connected clusters using the subsystem-as-IP pattern: a subsystem is 'just another IP' whose FSM processes are the glue between member IP models, and whose

resources are the member model instances. The subsystem template adds a subsystem: section declaring members and connections. Because the artifact shape is unchanged, every existing gate applies without tool changes, and the dedicated wiring gate verifies the declared composition points at real member APIs and glue FSMs. These subsystems expose integration behavior single-IP models cannot show: end-to-end latency across member pipelines, cross-IP backpressure, QoS interaction, and interrupt-storm dynamics.

5.1 dma_subsystem — data-mover workload (4 member IPs)

Members: gdma_ip (descriptor engine), axi_interconnect_ip (fabric router), arbitration_ip (downstream scheduler), completion_ip (QoS accounting).

```
Host Descriptor
  -> host_command          gdma.submit + READ/WRITE fabric commands
  -> axi_interconnect      route to slave, produce responses    [member]
  -> fabric_bridge         rebuild tenant/size, arbitration.enqueue
  -> arbitration_ip        port/tenant/SQ schedule + issue      [member]
  -> downstream_dispatch   completion.submit (QoS token path)
  -> completion_ip         token check/debit, emit              [member]
  -> completion_collector  both-legs match -> subsystem IRQ
  backpressure_monitor: fabric congestion -> gdma memory_ready,
                        completion backlog -> arbitration issue_ready
```

- A descriptor completes only when its engine leg (inside GDMA) and its fabric leg (both READ and WRITE accounted by completion_ip) have finished — then the subsystem IRQ fires.
- Workloads tested: single-descriptor end-to-end latency; fabric saturation throttling the engine (outstanding_limit=1) with all descriptors still completing; QoS token exhaustion delaying completion until refill; two channels mapping to two tenants completing independently.
- Channel N maps to tenant TN; the fabric bridge rebuilds arbitration commands from tracked descriptor state because interconnect responses drop tenant/size context.

5.2 mailbox_irq_subsystem — interrupt-delivery workload (2 member IPs)

Members: mailbox_ip (doorbell interrupt source), interrupt_controller_ip (delivery fabric). Closes the canonical doorbell round trip with a modeled software service loop.

```
Host Message
  -> host_message         mailbox.send_message (timestamped)
  -> mailbox_ip           push -> doorbell -> notify            [member]
  -> irq_source_bridge    consume doorbell, set_level(src, True)
  -> interrupt_controller pend -> filter -> priority -> deliver [member]
  -> cpu_service          controller.ack
  -> software_handler     read message, clear when drained, EOI
  storm_monitor: mask flooding source for a throttle window
```

- True level-triggered semantics: multiple doorbells on one channel collapse to a single pending controller entry; the handler keeps the level asserted while unserviced messages remain, so the controller's EOI reassert re-pends the source — one message per service round.
- Storm throttling is duty-cycle (mask for a time window, re-mask if still flooding) because pending collapse means a masked source can never drain below the limit on its own — a finding that mirrors how real interrupt-storm mitigation works.
- Workloads tested: single-message round trip (message-to-EOI latency); 3-message level-reassert recovery; storm throttle on a 4-message flood with eventual full service; masked-channel isolation with a healthy neighbor channel.

5.3 What integration exposed

- End-to-end descriptor latency through four member pipelines is a measured output, dominated by member timing rather than glue overhead (11 glue cycles per descriptor/message).
- Cross-IP backpressure works through member readiness setters: interconnect saturation deasserts GDMA memory readiness; completion backlog deasserts arbitration issue readiness.
- Member APIs were composed as-is — no member model was modified to build either subsystem; all integration lives in glue processes and the tracking tables they share.
- Composition needs its own gate: before check_subsystem_wiring.py, a typo'd member or renamed API would have passed every existing check silently.

6. Current Status

Measure	Value
Modeled artifacts	11 (9 IPs + 2 subsystems)
DLDs through the strict front-end gate	11 of 11
Unit tests	58, all passing
Validation tools	11 deterministic scripts in tools/
Top gate	python tools/validate_dld_flow.py — green
Authoring aids	templates/reference_template.yaml (annotated, lint-clean), prompt packs, skill + agent contract + harness

7. Next Steps (Candidates)

- Multi-source interrupt subsystem: timer, mailbox, and SPI sources feeding the interrupt controller, exercising cross-source priority interaction.
- Peripheral bus subsystem: axi_interconnect fanning out register traffic to SPI/I3C/timer slaves (contention, slow-slave backpressure).
- Performance experiments layer: sweep configurations (queue depths, storm windows, QoS budgets, outstanding limits) and emit latency/throughput reports from the existing models.
- SoC-level composition: subsystem-of-subsystems sharing one interrupt controller, testing whether the pattern nests.